ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

Compte rendu de TP

L'interface de démonstration Streamlit

Développement et déploiement d'applications intelligentes

Réalisé par
Othmane Moussawi
en binôme avec Ahmed Rayane Ramzi

Enseignant : A. Larhlimi

Année scolaire : 2026–2027
14 octobre 2026

Table des matières

1 Introduction **1**

2 Installation et lancement **1**
2.1 Démarche 1
2.2 Réalisation 1
2.3 Interprétation 1

3 Onglet « Un avis » **1**
3.1 Démarche 1
3.2 Réalisation 1
3.3 Interprétation 2

4 Onglet « Un fichier CSV » **2**
4.1 Démarche 2
4.2 Réalisation 2
4.3 Interprétation 2

5 Seuil d'incertitude **2**
5.1 Démarche 2
5.2 Réalisation 3
5.3 Interprétation 3

6 Tests **3**

1. Introduction

Ce TP rend le modèle du TP2 utilisable par quelqu'un qui n'ouvre jamais de terminal. L'interface Streamlit charge `models/sentiment.joblib` et appelle la fonction `predict()` du TP2 : aucune logique de prédiction n'est réécrite. Deux questions de conception sont au centre du TP : que montrer quand le modèle hésite, et quel chiffre l'équipe métier regarde en premier.

2. Installation et lancement

2.1 Démarche

Le kit `tp3-streamlit-kit` est installé dans le dépôt par `appliquer_tp3.py`, d'abord en simulation, puis pour de bon. L'interface est lancée avec `streamlit run ui/app.py`.

2.2 Réalisation

```
python appliquer_tp3.py ../sentiment-app
streamlit run ui/app.py
```

Contrôle S1 : la page s'ouvre sur `http://localhost:8501` avec les trois onglets « Un avis », « Un fichier CSV » et « Comment ça marche ». Le contrôle est **conforme**, vérifié dans le navigateur.

2.3 Interprétation

La page se charge sans erreur parce que le modèle est présent. Sans lui, `charger_predicteur()` lève une erreur explicite qui indique la commande à lancer, et la page s'arrête proprement au lieu de planter.

3. Onglet « Un avis »

3.1 Démarche

La logique est dans `ui/helpers.py` : `classer()` ajoute au résultat de `predict()` un verdict lisible, « à vérifier » lorsque la confiance est inférieure à `SEUIL_INCERTITUDE = 0.60`. Trois avis sont saisis, un net positif, un net négatif et un mitigé.

3.2 Réalisation

Résultats obtenus avec le modèle final, entraîné sur Allociné :

Avis	Label	Confiance	Verdict
Livraison rapide, produit conforme, je recommande.	positive	0,59	à vérifier
Produit cassé à l'arrivée et service client injoignable.	negative	0,58	à vérifier
Emballage soigné et notice claire, sauf que ensuite application qui plante.	negative	0,65	negative

TABLE 1 – Verdicts de l'onglet « Un avis » avec le modèle Allociné.

Contrôle S2 : **non conforme**. Le contrôle attend un verdict net pour les deux premiers avis, et « à vérifier » pour le troisième. Ici, les deux premiers sont signalés à vérifier, et le mitigé est tranché « négatif » à 65

Pour comparer, le modèle entraîné sur le jeu de démonstration (accuracy 0,879) a été utilisé brièvement : il donne 0,75 positif, 0,85 négatif, et « à vérifier » à 0,52 pour le mitigé, ce qui correspond à l'attendu. Ce test a été fait sur une copie temporaire ; le modèle Allociné a ensuite été restauré.

3.3 Interprétation

Le modèle Allociné a appris le vocabulaire des critiques de films (scénario, acteurs, mise en scène), et il connaît mal celui des avis produits (livraison, colis, service client). Il est donc peu sûr sur des avis courts de type e-commerce, même lorsqu'ils sont nets. Le seuil d'incertitude ne corrige pas ce décalage de domaine : seules des données étiquetées du bon domaine le feraient. Ce point est présenté comme un échec réel, et non comme un bug de l'interface.

4. Onglet « Un fichier CSV »

4.1 Démarche

`predire_lot()` écarte les lignes vides, refuse une colonne `review` absente avec un message explicite, puis `resumer()` calcule les métriques produit : nombre d'avis, part de négatifs et nombre d'avis à vérifier.

4.2 Réalisation

Le contrôle S3 demande de téléverser `data/demo_reviews.csv` dans l'interface. **Ce téléversement n'a pas été fait dans le navigateur** ; nous avons seulement vérifié les fonctions de calcul sur ce fichier, avec le modèle Allociné :

Seuil	Avis	Négatifs détectés (%)	À vérifier
0,50	290	210 (72,4 %)	0
0,60	290	210 (72,4 %)	117
0,75	290	210 (72,4 %)	234

Table 2 – Résumé du fichier de démonstration selon le seuil.

Le fichier contient en réalité 145 négatifs sur 290, soit 50

4.3 Interprétation

L'interface annonce 72,4

5. Seuil d'incertitude

5.1 Démarche

Le seuil a été évalué sur les 4 000 avis de test Allociné, pour cinq valeurs entre 0,50 et 0,75. Pour chacune, on compte les avis envoyés à un agent et les négatifs classés positifs sans relecture.

5.2 Réalisation

Seuil	Avis envoyés à un agent	Erreurs tranchées seules	Négatifs manqués
0,50	0 (0 %)	318	181
0,55	249 (6,2 %)	206	118
0,60	**509 (12,7 %)**	**126**	**74**
0,65	846 (21,1 %)	76	43
0,70	1 183 (29,6 %)	49	28
0,75	1 626 (40,6 %)	27	17

TABLE 3 – Effet du seuil d'incertitude sur le jeu de test Allociné (4 000 avis).

5.3 Interprétation

Nous retenons **0,60**. Ce seuil divise par 2,4 le nombre de négatifs que le modèle laisse passer sans relecture (de 181 à 74), en échange d'environ un avis sur huit. Passer à 0,75 évite encore 57 négatifs manqués, mais envoie 1 117 avis de plus à la file humaine. Ce choix doit être arbitré avec l'équipe relation client, car il dépend du coût réel d'un client mécontent oublié.

Réponse

Seuil retenu : 0,60. Sur le jeu de test, 509 avis sur 4 000 (12,7

6. Tests

Le test d'interface `tests/test_ui.py` passe (3 tests). Sur l'ensemble du dépôt, `pytest -q` donne **44 passed, 1 failed** : le test en échec est l'historique Git, qui attend le commit du second membre. Le contrôle S4 attend 45 passed après ce commit.

Conclusion

L'interface se lance et le code de prédiction reste celui du TP2. Le contrôle S2 révèle un décalage de domaine entre les critiques de films et les avis produits : c'est le résultat le plus important du TP, et il montre qu'une bonne mesure n'est valable que sur des données semblables à celles de production. Le contrôle S3 reste à faire dans le navigateur.